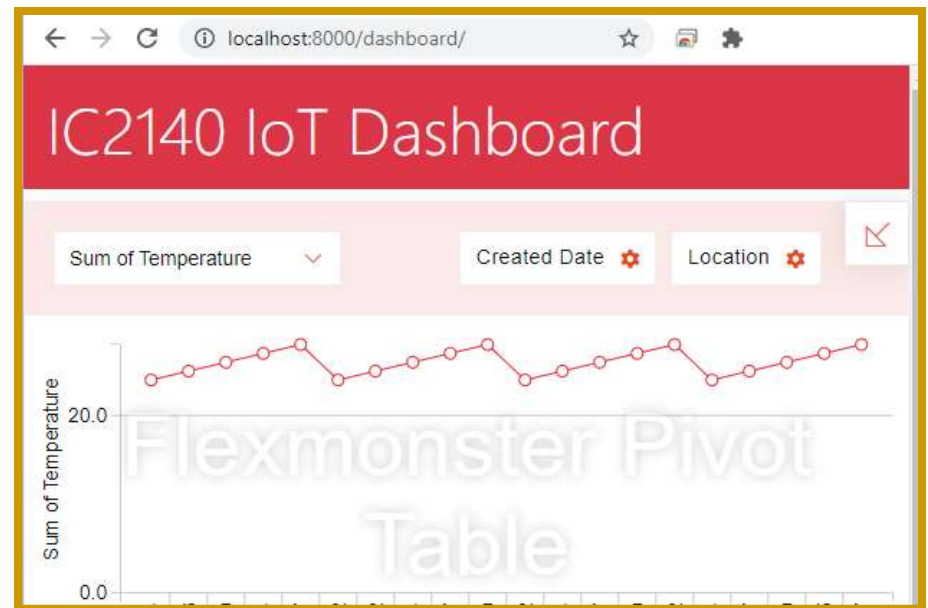

Advanced System Applications and Practice
for Internet and Multimedia
TM1117
Django Dashboard App

Instructor :

**Industrial Centre
The Hong Kong Polytechnic University**

Example: Create IoT Dashboard

- Plot the temperature in line chart
- Customize the appearance of dashboard
- Automatically update the dashboard



Create "dashboard" application(app)

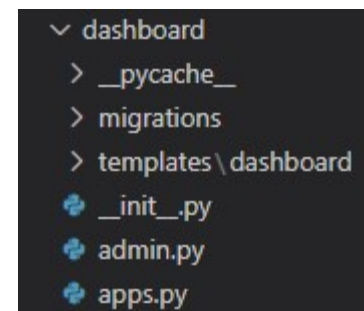
- 1) Shutdown the django server in the Anaconda Prompt
 - a) Press CTRL + C
- 2) Create "dashboard" application(app)
\$ python manage.py startapp dashboard
- 3) Register the "dashboard" app to the project in Visual Studio Code
 - a) Open the mysite\mysite\settings.py file in the Visual Studio Code
 - b) Search the "INSTALLED_APPS" location
 - c) Add the "dashboard" app to "INSTALLED_APPS" variable

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'blog',  
    'iot',  
    'dashboard',  
]
```



Create Dashboard index page (home page)

- 1) Create a folder "templates" under the `mysite/dashboard` folder in the Visual Studio Code
 - a) Right-click the "dashboard" folder in Visual Studio Code
 - b) Select the "New folder" and assign a name "templates" to it
- 2) Create a folder "dashboard" under the `mysite/dashboard/templates` folder in the Visual Studio Code
 - a) Right-click the "templates" folder in Visual Studio Code
 - b) Select the "New folder" and assign a name "dashboard" to it



Create Dashboard index page (home page)

- 3) Create a file "index.html" under the [mysite/dashboard/templates/dashboard](#) folder
 - a) Right-click the "dashboard" folder in Visual Studio Code
 - b) Select the "New file" and assign a name "index.html" to it
- 4) Put the below HTML code (static page) to [mysite/dashboard/templates/dashboard/index.html](#) file

```
<!doctype html>
<HTML>
  <head>
    <title>IC2140 IoT Dashboard</title>
  </head>
  <body>
    <h1>IC2140 IoT Dashboard</h1>
  </body>
</HTML>
```

Render Dashboard index page (home page)

- 1) Render the "index.html" page when requested by user
 - a) Open the [mysite/dashboard/views.py](#) file in the Visual Studio Code
 - b) Add the following code in it

```
def index(request):  
    return render(request, 'dashboard/index.html')
```

Map URL to Dashboard index page (home page)

- 1) Create a file "urls.py" under the [mysite/dashboard/](#) folder
 - a) Right-click the dashboard folder in Visual Studio Code
 - b) Select the "New file" and assign a name "urls.py" to it
 - c) Paste the following code in it

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.index),
]
```

Map URL to dashboard index page (home page)

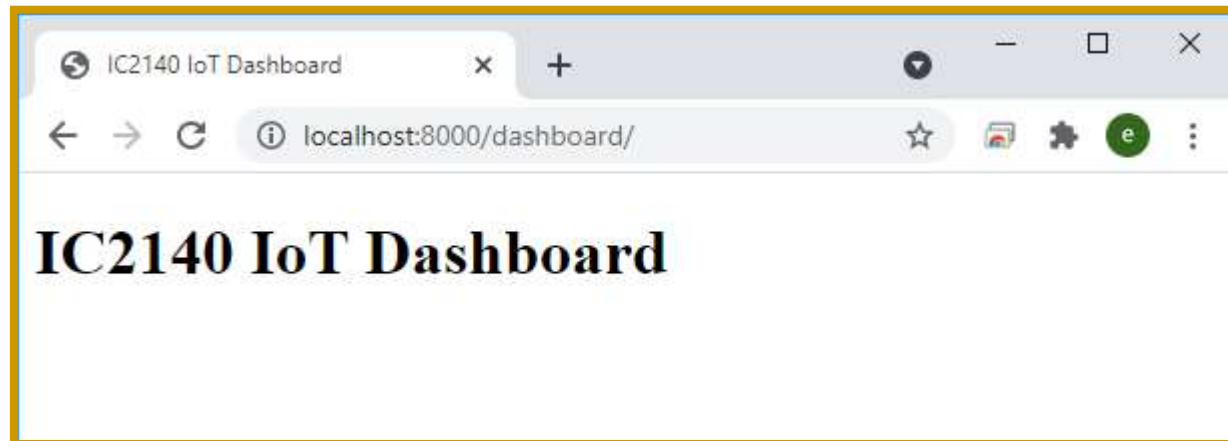
- 2) Update the routing in [mysite/mysite/urls.py](#) file (Entry point of the Project)
 - a) Open the [mysite/mysite/urls.py](#) in the Visual Studio Code
 - b) Append the "urlpatterns" variable with the following code (**Brown arrow**)

```
urlpatterns = [  
    path('admin/', admin.site.urls),  
    path('', include('blog.urls')),      #Include app "blog/urls.py"  
    path('iot/', include('iot.urls')),   #Include app "iot/urls.py"  
    path('dashboard/', include('dashboard.urls')), #Include app "dashboard/urls.py"  
]
```



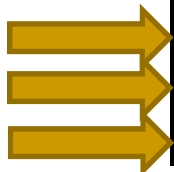
Access the Dashboard index page in browser

- 1) Start the server in the Anaconda Prompt (if stop)
\$ python manage.py runserver
- 2) Open the link as below in the Chrome Browser
<http://localhost:8000/dashboard/>



Extract "Event" data from database

- 1) Import necessary packages (JsonResponse and serializers) and the model class that we can access data "Event" in the views
 - a) Open the [mysite/dashboard/views.py](#) file in the Visual Studio Code
 - b) Add the 2nd, 3rd & 4th statements (**Brown arrow**) in it



```
from django.shortcuts import render
from django.http import JsonResponse
from django.core import serializers
from iot.models import Event
```

Extract "Event" data from database

- 2) Retrieve all events from the database
 - a) Open the [mysite/dashboard/views.py](#) file in the Visual Studio Code
 - b) Create a `temp_data()` method with the following code

```
def temp_data(request):  
    events = Event.objects.all()  
    data = serializers.serialize('json', events) #Translating Django models into JSON formats  
    return JsonResponse(data, safe=False)        #Returns a string that contains an array object
```

Map URL to Dashboard temp_data() method

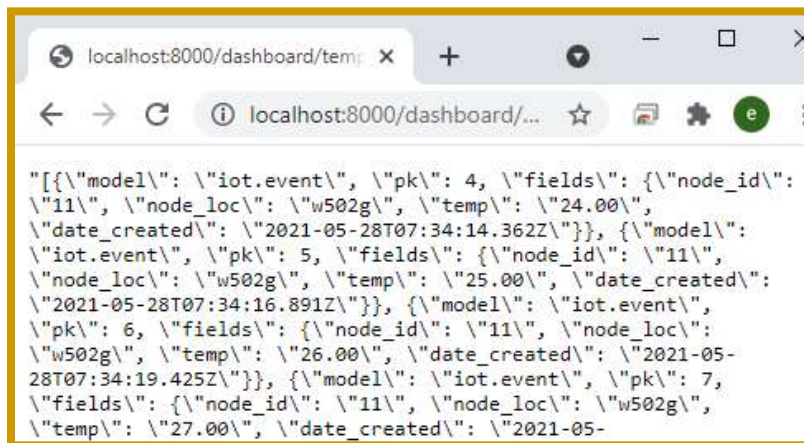
- 1) Update the routing in [mysite/dashboard/urls.py](#) file (Entry point of the temp_data())
 - a) Open the [mysite/dashboard/urls.py](#) file in the Visual Studio Code
 - b) Append the "urlpatterns" variable with the following code (Brown arrow)



```
urlpatterns = [  
    path('', views.index),  
    path('temp_data', views.temp_data, name='temp_data'),  
]
```

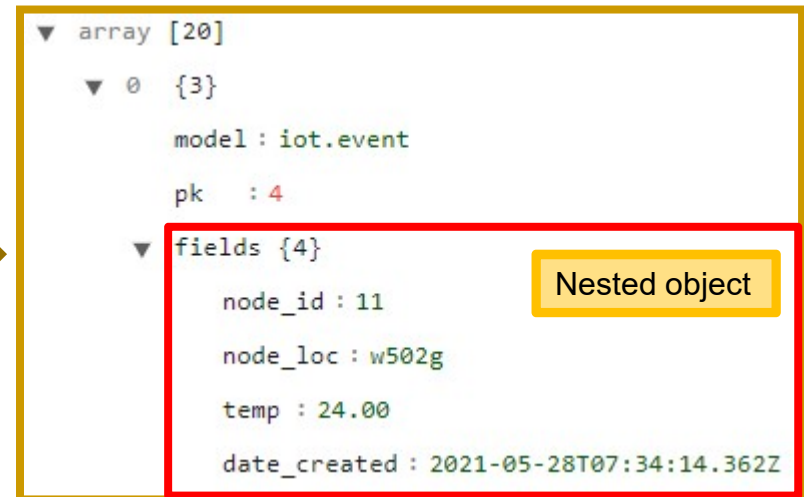
Access the Dashboard temp_data() method in browser

- 1) Start the server in the Anaconda Prompt (if stop)
\$ python manage.py runserver
- 2) Open the link as below in the Chrome Browser (if success, the JSON object is returned)
http://localhost:8000/dashboard/temp_data



localhost:8000/dashboard/temp_data

```
[{"model": "iot.event", "pk": 4, "fields": {"node_id": "11", "node_loc": "w502g", "temp": "24.00", "date_created": "2021-05-28T07:34:14.362Z"}}, {"model": "iot.event", "pk": 5, "fields": {"node_id": "11", "node_loc": "w502g", "temp": "25.00", "date_created": "2021-05-28T07:34:16.891Z"}}, {"model": "iot.event", "pk": 6, "fields": {"node_id": "11", "node_loc": "w502g", "temp": "26.00", "date_created": "2021-05-28T07:34:19.425Z"}}, {"model": "iot.event", "pk": 7, "fields": {"node_id": "11", "node_loc": "w502g", "temp": "27.00", "date_created": "2021-05-28T07:34:21.959Z"}}
```



array [20]

- 0 {3}
- model : iot.event
- pk : 4
- fields {4}
- node_id : 11
- node_loc : w502g
- temp : 24.00
- date_created : 2021-05-28T07:34:14.362Z

Nested object

Display JSON Events in the Dashboard

- 1) Include Flexmonster from CDN in the HTML page
 - a) Open the [mysite/dashboard/templates/dashboard/index.html](#) file
 - b) Place the following code (**Brown arrow**) after `<h1>.....</h1>` block



```
<h1>IC2140 IoT Dashboard</h1>
<script src="https://cdn.flexmonster.com/flexmonster.js"></script>
<script src="https://code.jquery.com/jquery-3.3.1.min.js"></script>
```

Display JSON Events in the Dashboard

2) Add a container for Flexmonster

- a) Use a <div> container for the component
- b) Place the following code (**Brown arrow**) after <script>.....</script> block

```
<h1>IC2140 IoT Dashboard</h1>
<script src="https://cdn.flexmonster.com/flexmonster.js"></script>
<script src="https://code.jquery.com/jquery-3.3.1.min.js"></script>
<div id="pivot-container" ></div>
```



<div></div>

- Defines a division or a section in an HTML document.

Display JSON Events in the Dashboard

- 3) Since the JSONResponse contains an array object with extra meta information (see slide 13), we need to do the data processing on the client side (front end). Flexmonster only accepts an array of JSON objects without nested level. Therefore, We add one function processData() to extract only those nested object and put them into a single array.
- a) Place the following code after <div>.....</div> block

```
<script>
    function processData(dataset) {
        var result = []
        dataset = JSON.parse(dataset);
        dataset.forEach(item => result.push(item.fields));
        return result;
    }
</script>
```


Display JSON Events in the Dashboard

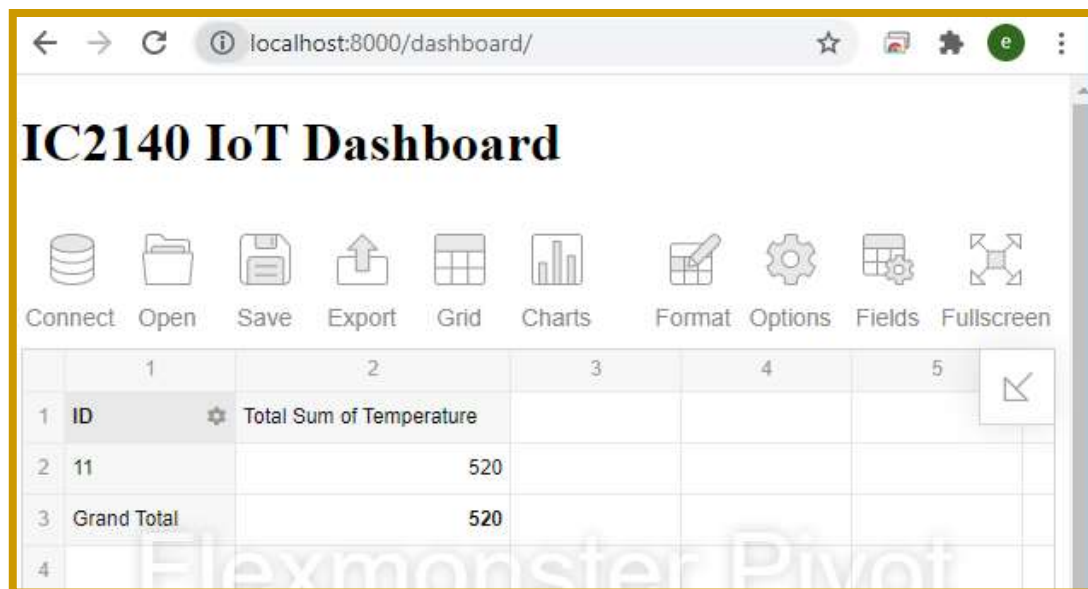
- 4) Add a simple script to embed the component into the web page
 - a) Place the following code after <script>.....</script> block

```
<script>
    $.ajax({
        url: '/dashboard/temp_data',
        dataType: 'json',
        success: function(data) {
            new Flexmonster({
                container: "#pivot-container",
                componentFolder: "https://cdn.flexmonster.com/",
                toolbar: true,
                report: {
                    dataSource: {
                        type: "json",
                        data: processData(data),
                        mapping: {
                            "node_id": { "caption": "ID" },
                            "node_loc": { "caption": "Location" },
                            "temp": { "caption": "Temperature", "type": "number" },
                            "date_created": { "caption": "Created Date", "type": "datetime" }
                        }
                    },
                },
            });
        }
    });
</script>
```

Display JSON Events in the Dashboard

5) Open the link as below in the Chrome Browser

a) <http://localhost:8000/dashboard/>



Configure the report to show the subset of data

- 1) The slice properties is used to define the row, column and measure in the dashboard report
 - a) Place the following code after `dataSource: {.....}` block within `report: {.....}`

```
slice: {  
  rows: [  
    { uniqueName : "date_created"},  
  ],  
  columns: [  
    { uniqueName : "node_loc"},  
    { uniqueName : "[measures]", label : "Temperature" },  
  ],  
  measures: [  
    { uniqueName : "temp" },  
  ]  
},
```

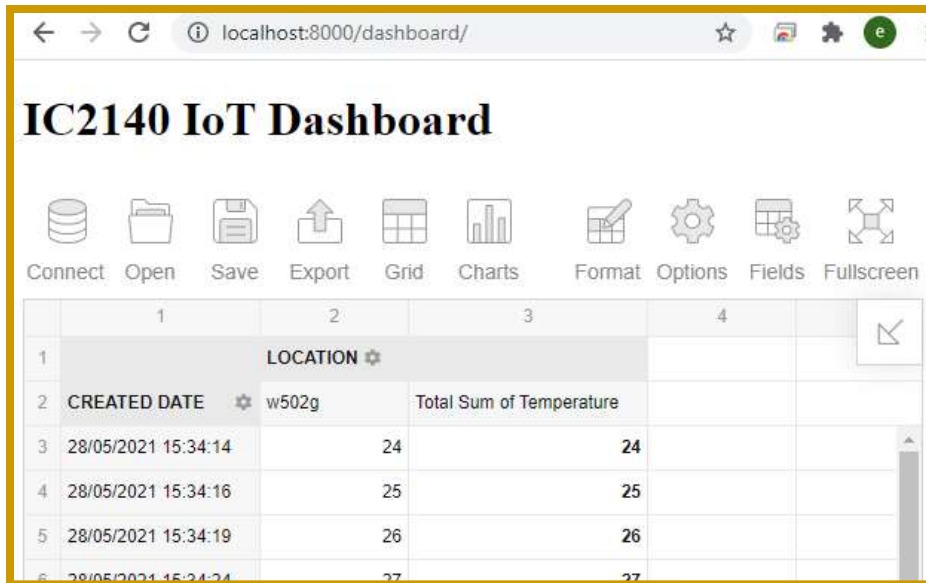
Row: date_created
Column: node_loc & temp

Reference: <https://www.flexmonster.com/doc/slice/>

Configure the report to show the subset of data

2) Open the link as below in the Chrome Browser

a) <http://localhost:8000/dashboard/>



The screenshot shows the IC2140 IoT Dashboard in a Chrome browser. The dashboard has a toolbar with icons for Connect, Open, Save, Export, Grid, Charts, Format, Options, Fields, and Fullscreen. Below the toolbar is a table with the following data:

	1	2	3	4
1	LOCATION			
2	CREATED DATE	w502g	Total Sum of Temperature	
3	28/05/2021 15:34:14	24	24	
4	28/05/2021 15:34:16	25	25	
5	28/05/2021 15:34:19	26	26	
6	28/05/2021 15:34:24	27	27	

Set 1 decimal place for temperature

- 1) Create the format "degree" in the dashboard report
 - a) Place the following code after slice: {...} block within report: {...}

```
formats: [  
  {  
    name: "degree",  
    decimalPlaces: 1,  
  }  
],
```

Set 1 decimal place for temperature

- 2) Attach the format "degree" to the measures option for temperature
 - a) Append the "measures" option with the following code (Brown arrow)

```
measures: [  
  {  
    uniqueName : "temp",  
    format : "degree",  
  }  
]
```



Set 1 decimal place for temperature

3) Open the link as below in the Chrome Browser

a) <http://localhost:8000/dashboard/>

	1	2	3	4
1	LOCATION			
2	CREATED DATE	w502g	Total Sum of Temperature	
3	28/05/2021 15:34:14	24.0	24.0	
4	28/05/2021 15:34:16	25.0	25.0	
5	28/05/2021 15:34:19	26.0	26.0	
6	28/05/2021 15:34:24	27.0	27.0	

Indicate the temperature below the threshold using conditional formatting

- 1) Create the conditional formatting and apply to the temp measures option in the dashboard report
 - a) Place the following code after format: [.....] block within report: {.....}

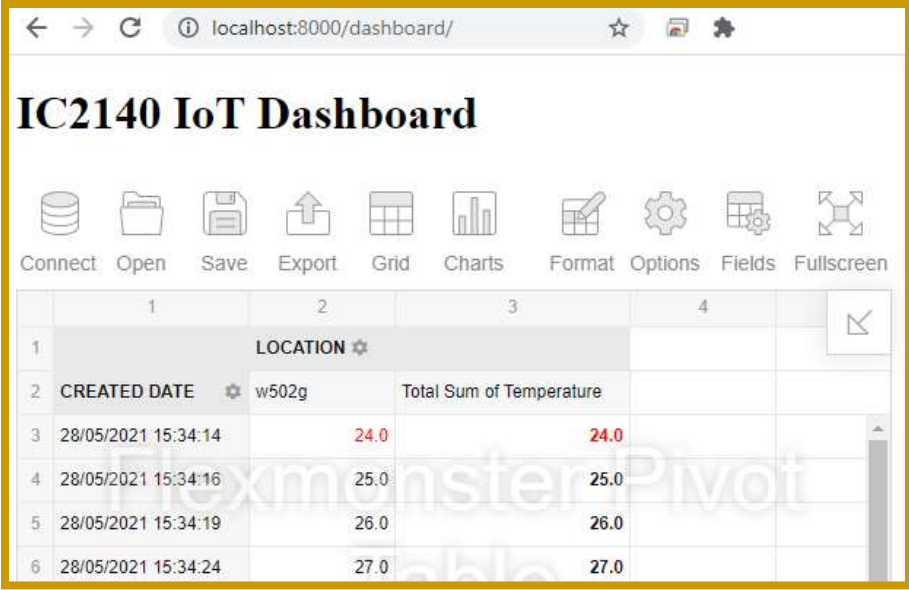
```
conditions: [  
  {  
    formula: "#value < 25",  
    measure: "temp",  
    format: {  
      "color": "#FF0000"  
    }  
  }  
],
```

Show RED colour if
the temperature is
lower than 25

Indicate the temperature below the threshold using conditional formatting

2) Open the link as below in the Chrome Browser

a) <http://localhost:8000/dashboard/>

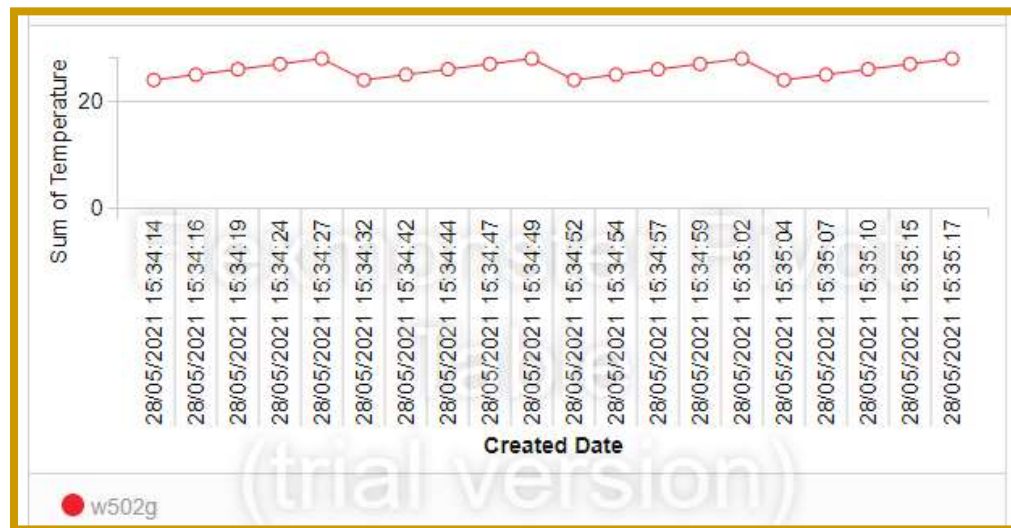


The screenshot shows a web browser window with the address bar displaying `localhost:8000/dashboard/`. The page title is "IC2140 IoT Dashboard". Below the title is a toolbar with icons for Connect, Open, Save, Export, Grid, Charts, Format, Options, Fields, and Fullscreen. The main content area displays a table with the following data:

	1	2	3	4
1	LOCATION ⚙️			
2	CREATED DATE ⚙️	w502g	Total Sum of Temperature	
3	28/05/2021 15:34:14	24.0	24.0	
4	28/05/2021 15:34:16	25.0	25.0	
5	28/05/2021 15:34:19	26.0	26.0	
6	28/05/2021 15:34:24	27.0	27.0	

Display the report in Line Chart

- 1) The grid is the default view shown by the component. To display the line chart, we can switch it using the Toolbar
 - a) Click the Charts icon  on the Toolbar
 - b) Select Line icon  Line



Set the Line Chart as the default view

- 1) Change the grid to the chart by using viewType option
 - a) Disable the toolbar with two forward slashes

```
//toolbar: true,
```
 - b) Place the following code after conditions: [.....] block within report: {.....}

```
options: {  
  viewType: "charts",  
  chart: {  
    type: "line",  
  },  
},
```

Set the Line Chart as the default view

2) Open the link as below in the Chrome Browser

a) <http://localhost:8000/dashboard/>



Flexmonster Themes

■ Built-in CSS themes

Flexmonster comes with a variety of predefined CSS themes:

- Striped-Blue
- Striped-Teal
- Purple
- Black-Orange
- Bright-Orange
- Yellow
- Green
- Midnight
- Mac OS
- Soft-Default
- Accessible (compatible with high contrast modes)
- Light blue
- Dark
- Teal
- Orange
- Default
- 2.3-styled (for those who enjoyed the style of major version 2.3)

Reference: <https://www.flexmonster.com/doc/customizing-appearance/>

Apply Flexmonster theme in the dashboard

1) Apply "orange" theme

- a) Add the references to the CSS of the chosen theme by putting the following code (Brown arrow)



```
<h1>IC2140 IoT Dashboard</h1>  
<script src="https://cdn.flexmonster.com/flexmonster.js"></script>  
<script src="https://code.jquery.com/jquery-3.3.1.min.js"></script>  
<link rel="stylesheet" type="text/css" href="https://cdn.flexmonster.com/theme/orange/flexmonster.min.css" />
```

- b) To apply other theme, replace the "orange" with the chosen name
- 

Apply Bootstrap CSS for title

- 1) Add the following code inside <head> section before the <title>...</title> tag

```
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.0.0/dist/css/bootstrap.min.css" rel="stylesheet" integrity="sha384-wEmeIV1mKuiNpC+IOBjI7aAzPcEZeedi5yW5f2yOq55WWLwNGmvvx4Um1vskeMj0" crossorigin="anonymous">
```

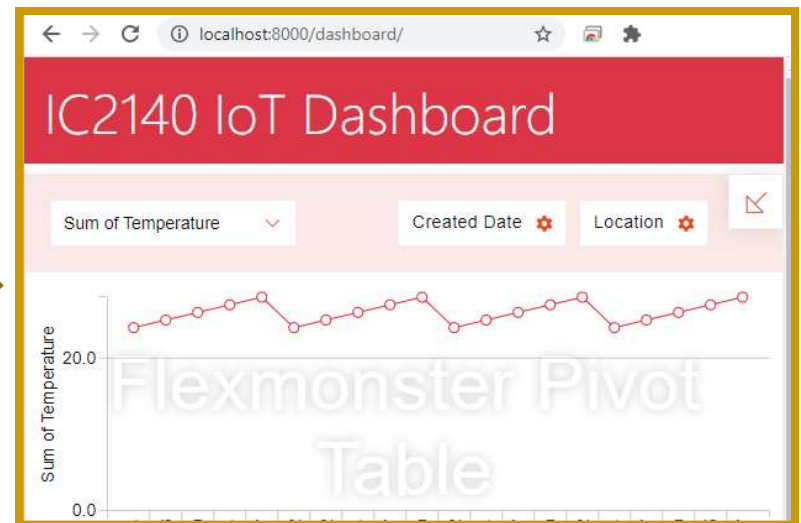
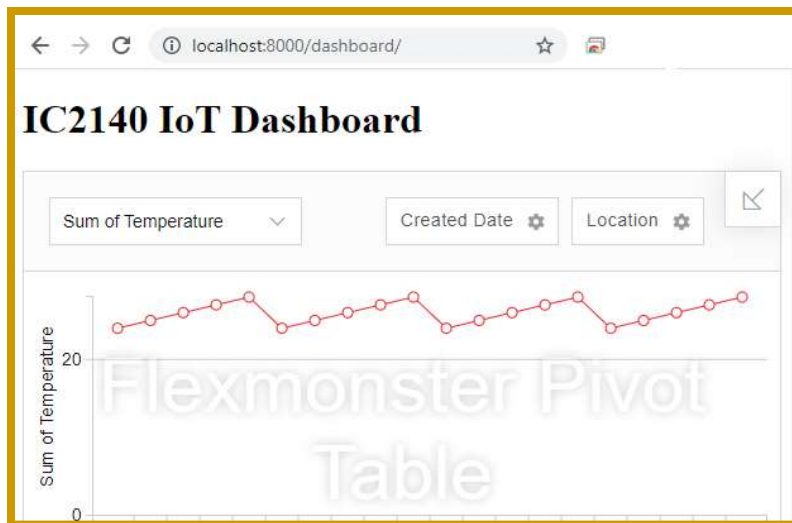
- 2) Replace the <h1>...</h1> code block with the following code

```
<h1 class="display-3 p-3 mb-2 bg-danger text-white">IC2140 IoT Dashboard</h1>
```

Apply Bootstrap CSS for title

3) Open the link as below in the Chrome Browser

a) <http://localhost:8000/dashboard/>



Update the dashboard every 5 seconds

- To avoid the reloading the whole web page, the Flexmonster `updateData()` function is used to add or remove some data inside the Flexmonster component.

```
flexmonster.updateData({  
    data: processData(data)  
});
```

- To send AJAX request at regular interval (E.g. 5 seconds), the `setTimeout()` function is used. It can ensure that the last request is completed before sending the next one.

```
setTimeout(getFeed, 5000);    // 5 seconds
```

Update the dashboard every 5 seconds

- 1) Integrate the `updateData()` and `setTimeout` functions into the dashboard webpage
 - a) Place the following code after `<script>.....</script>`

```
<script>
function getFeed() {
    $.ajax({
        url: '/dashboard/temp_data',
        dataType: 'json',
        success: function(data) {
            flexmonster.updateData({
                data: processData(data)
            });
        },
        complete: function(data) {
            setTimeout(getFeed, 5000);
        }
    });
}

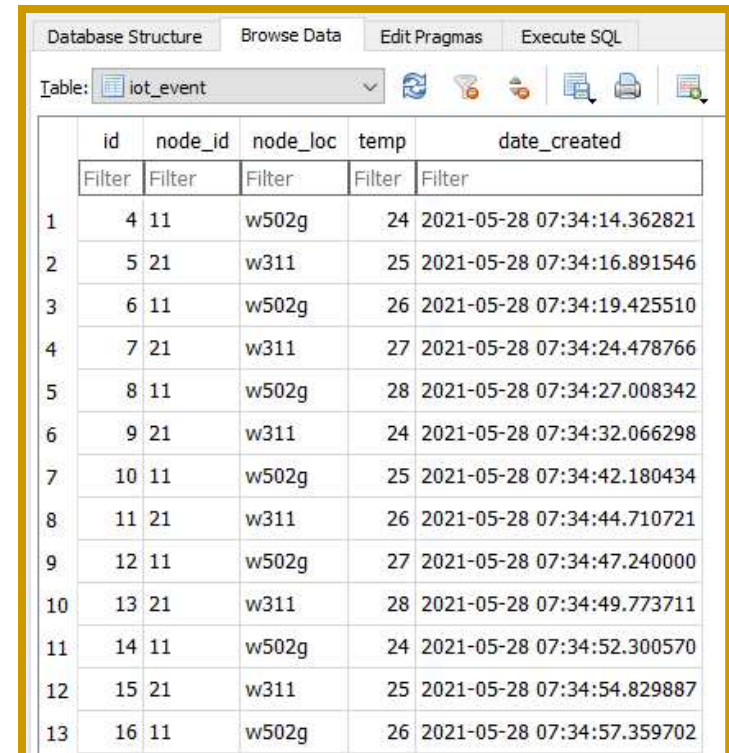
$(document).ready(function() {
    setTimeout(getFeed, 5000);    // 5 seconds
});
</script>
```

Django Dashboard Assignment 1 :

- Complete whole Dashboard application (Slide 3 to Slide 34) and add the following requirements
 - 1) Apply 2 decimal places for the temperature
 - 2) Update the data in run time every 10 seconds
 - 3) Simplify the dashboard layout by removing the "Created Date" and "Location" filter
 - 4) Set the title to "IoT Sensor"
 - 5) Apply Flexmonster "Dark" CSS Theme
 - 6) Set the Y-axis label to "Temperature"
 - Show the demonstration with suggested testing
-

Django Dashboard Assignment 2 :

- Assume that the database contains the following data
 - ❑ Sensor ID: 11 & 21
 - ❑ Location: w502g & w311
 - ❑ Temperature: Any value



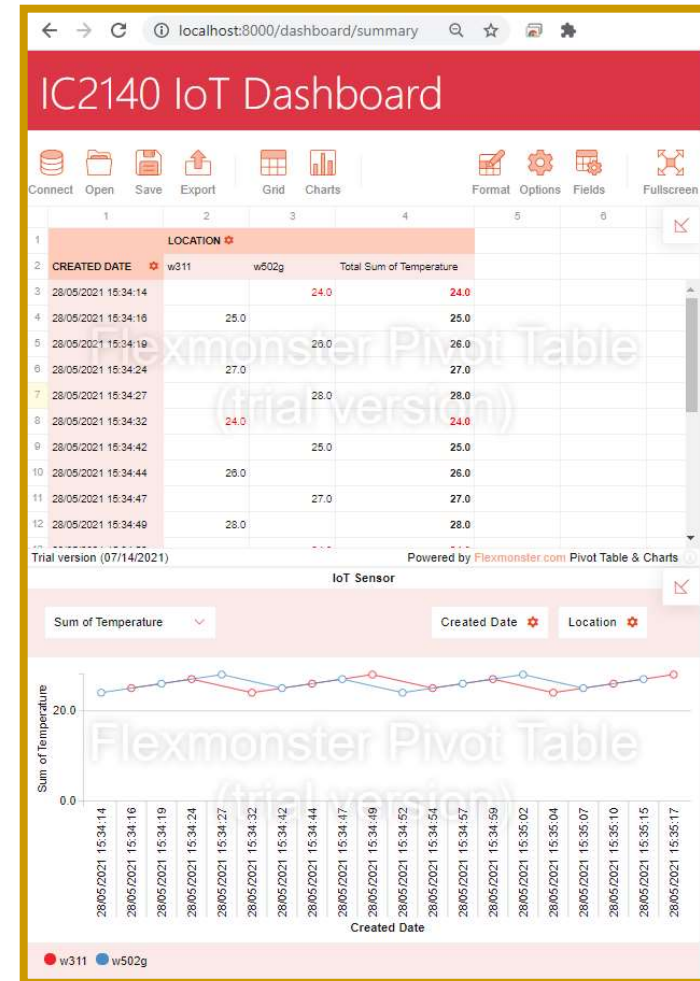
The screenshot shows a database management interface with tabs for 'Database Structure', 'Browse Data', 'Edit Pragma', and 'Execute SQL'. The 'Browse Data' tab is active, displaying a table named 'iot_event'. The table has five columns: 'id', 'node_id', 'node_loc', 'temp', and 'date_created'. Each column has a 'Filter' button above it. The table contains 13 rows of data, with 'id' ranging from 1 to 13, 'node_id' alternating between 11 and 21, 'node_loc' alternating between 'w502g' and 'w311', 'temp' ranging from 24 to 28, and 'date_created' showing timestamps from 2021-05-28 07:34:14.362821 to 2021-05-28 07:34:57.359702.

	id	node_id	node_loc	temp	date_created
	Filter	Filter	Filter	Filter	Filter
1	4	11	w502g	24	2021-05-28 07:34:14.362821
2	5	21	w311	25	2021-05-28 07:34:16.891546
3	6	11	w502g	26	2021-05-28 07:34:19.425510
4	7	21	w311	27	2021-05-28 07:34:24.478766
5	8	11	w502g	28	2021-05-28 07:34:27.008342
6	9	21	w311	24	2021-05-28 07:34:32.066298
7	10	11	w502g	25	2021-05-28 07:34:42.180434
8	11	21	w311	26	2021-05-28 07:34:44.710721
9	12	11	w502g	27	2021-05-28 07:34:47.240000
10	13	21	w311	28	2021-05-28 07:34:49.773711
11	14	11	w502g	24	2021-05-28 07:34:52.300570
12	15	21	w311	25	2021-05-28 07:34:54.829887
13	16	11	w502g	26	2021-05-28 07:34:57.359702

Cont.

Django Dashboard Assignment 2 :

- Develop dashboard to display the IoT sensor information in terms of pivot table and line chart



Cont.

Django Dashboard Assignment 2 :

- Allow user to access it using the below link
 - <http://localhost:8000/dashboard/summary>
- Complete the following requirements
 - 1) List the IoT sensor data including the sensor ID, location and etc.
 - 2) Show the summary of average, maximum, minimum of temperature in Table. Use filter() and aggregate() function
 - 3) Display the pivot table and Line Chart
 - 4) Indicate the warning if the temperature is higher than 30 degree with Blue colour
 - 5) Update the Flexmonster data in run time every 3 seconds
 - 6) Suggest any additional components